DOCUMENTATION · INSTALLATION

Oryon
Guide local

Installation, configuration et lancement du projet sur votre PC — base de données vide, données 100 % personnelles, zéro mock.

VERSION	DATE	COMMIT
1.0	2026-10-19	8b9269c

STACK
pnpm 12.4.2

SECTION 1

Vue d'ensemble & architecture

Oryon (nom de code interne **Cosmoz**) est une plateforme de trading analytics en **monorepo pnpm + Turborepo**. Un frontend Next.js, une API NestJS et des micro-services optionnels coopèrent autour d'un contrat de données partagé (`packages/shared` , schemas Zod) : types, primitives monétaires (decimal-string, UTC strict) et utilitaires de sécurité (assainissement CSV).

Composant	Stack	Port dev	Dossier
Web (frontend)	Next.js App Router + Tailwind + Radix + TanStack Query v5	`3000`	`apps/web`
API (backend)	NestJS + Prisma + class-validator	`4000`	`apps/api`
Base de données	PostgreSQL 17 embarquée (dev)	`5433`	`apps/api/.localpg`
OCR service *(optionnel)*	FastAPI (Python)	`3004`	`mini-services/ocr-service`
Market service *(optionnel)*	Bun sidecar	`3003`	`mini-services/market-service`

Flux en mode réel : le navigateur parle au web (:3000) ; les appels API partent vers le même origine sous `/api/v1` et sont proxifiés vers l'API (:4000). Aucun CORS à gérer en local, cookie de refresh first-party. Le backend MSW simulé est **désactivé** dans la configuration livrée — vous tapez toujours sur la vraie API et votre vraie base.

> **Base vide par design**
>
> Aucun seed, aucun jeu de données fictif, aucun compte de démo n'existe dans le dépôt. Après migration du schéma, toutes les tables sont vides : vous remplissez la base avec vos propres données via l'interface (Section 8).

SECTION 2

Prérequis

Outil	Version	Vérification	Notes
Node.js	≥ 20 (testé v24.21.0)	`node -v`	LTS recommandé
pnpm	**12.4.2** exactement	`pnpm -v`	via corepack (ci-dessous)
Git	récent	`git -v`	clonage / historique
PostgreSQL 17	option B seulement	—	inutile avec la DB embarquée

Python 3.11+	optionnel	`python3 --version`	service OCR
Bun	optionnel	`bun -v`	market service

```
# pnpm via corepack — aucune installation globale
corepack enable
corepack prepare pnpm@12.4.2 --activate
pnpm -v   # → 12.4.2
```

Le lockfile `pnpm-lock.yaml` est canonique (`bun.lock` a été retiré lors de la purge du 2026-10-19). N'utilisez ni npm ni yarn : pnpm garantit l'isolation des workspace packages et l'application des `overrides` de sécurité (deepmerge-ts, source-map-js, sharp épinglés → `pnpm audit --prod` = 0 vulnérabilité).

SECTION 3

Récupération du projet

Option A — Tarball clé en main (recommandé)

```
mkdir -p ~/oryon && tar -xzf oryon-full-workspace-FINAL-2026-10-19.tar.gz -
C ~/oryon
cd ~/oryon
git log --oneline -1   # doit afficher le commit de référence
```

Option B — Bundle git (historique complet)

```
git clone oryon-git-history-FINAL-2026-10-19.bundle oryon
cd oryon && git checkout main
# variante légère sans tout l'historique :
git clone --depth 1 oryon-git-history-FINAL-2026-10-19.bundle oryon
```

SECTION 4

Installation des dépendances

```
pnpm install --frozen-lockfile
```

Une seule commande installe tout : web, api, `packages/shared`, et génère le client Prisma (postinstall). Comptez 1 à 3 minutes sur une machine neuve. Ne retirez `--frozen-lockfile` que si vous ajoutez volontairement des dépendances.

SECTION 5

Variables d'environnement

Deux fichiers à créer à partir des templates documentés (`apps/api/.env.example` , `apps/web/.env.example`). **Aucun** `.env` **n'est versionné** — vérifié par la garde CI `check-tracked-secrets` (1528 fichiers scannés).

5.1 API — apps/api/.env (variables obligatoires)

Variable	Rôle	Génération
`DATABASE_URL`	Connexion PostgreSQL	`postgresql://cosmoz@127.0.0.1:5433/cosmoz` (DB embarquée, trust localhost)
`AUTH_JWT_SECRET`	Signature des JWT	`randomBytes(48).toString('base64url')`
`AUTH_MFA_ENCRYPTION_KEY`	AES-256-GCM secrets TOTP	`randomBytes(32).toString('base64')`
`INTEGRATIONS_WEBHOOK_ENCRYPTION_KEY`	AES-256-GCM secrets webhook	idem — clé distincte
`ACCOUNTS_CREDENTIALS_ENCRYPTION_KEY`	AES-256-GCM credentials brokers	idem — clé distincte

```
# Génération des secrets (exécutez 4 fois avec des tailles/variantes)
node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
```

Variable	Valeur locale	Rôle
`NODE_ENV`	`development`	Mode courant
`PORT`	`4000`	Port d'écoute API
`CORS_ORIGINS`	`http://localhost:3000`	Allow-list stricte (`*` rejeté au boot)
`LOG_LEVEL`	`debug`	Verbosité des logs
`TRUST_PROXY`	`false`	`true` uniquement derrière un proxy de confiance
`WEB_APP_URL`	`http://localhost:3000`	Base des liens email

Secrets déjà générés pour vous

Le tarball livre `apps/api/.env` avec 4 secrets CSPRNG frais (2026-10-19). Régénérez-les quand vous voulez avec les commandes ci-dessus.

5.2 Web — apps/web/.env.local

```
NEXT_PUBLIC_API_MOCKED=false
API_PROXY_TARGET=http://localhost:4000
```

⚠ NE PAS omettre NEXT_PUBLIC_API_MOCKED=false

Sans cette ligne, le mode dev Next.js reste sur le backend MSW simulé (données fictives). Avec `false`, toutes les requêtes partent sur votre vraie API via le proxy same-origin — zéro mock. En build production, le mode réel est imposé.

SECTION 6

Base de données — vide par design

Option A — PostgreSQL embarquée (recommandée en local)

```
pnpm api:db:up                          # démarre PostgreSQL 17 sur
127.0.0.1:5433
pnpm --filter @cosmoz/api db:status     # vérifie l'état
pnpm --filter @cosmoz/api exec prisma migrate deploy   # applique les 42
migrations
```

Binaire géré par la dépendance `embedded-postgres` — aucune installation système. Authentification trust sur 127.0.0.1 uniquement (convenience dev). Les données vivent dans `apps/api/.localpg/` (créé au premier `db:up`, hors git). `db:down` stoppe, `db:status` diagnostique.

Option B — Votre propre PostgreSQL (Docker)

```
docker run -d --name oryon-pg -e POSTGRES_USER=oryon \
  -e POSTGRES_PASSWORD=change-me -e POSTGRES_DB=cosmoz \
  -p 5433:5432 postgres:17
# puis adaptez DATABASE_URL dans apps/api/.env :
# postgresql://oryon:change-me@127.0.0.1:5433/cosmoz
pnpm --filter @cosmoz/api exec prisma migrate deploy
```

Pourquoi la base sera vide — et c'est voulu : aucun fichier de seed n'existe (aucune clé `prisma.seed`, aucun `prisma/seed.ts`) ; les 42 migrations créent uniquement la structure

(tables, index, contraintes uniques). Les données de test du développement vivaient dans `.localpg` (non versionné, non livré). Repartir de zéro plus tard : `db:down` → supprimez `apps/api/.localpg/` → `db:up` → `migrate deploy`.

SECTION 7

Lancement du projet

1. **Démarrer la base**

```
pnpm api:db:up
```

2. **Tout lancer (recommandé)**

```
pnpm dev   # turbo dev : API :4000 + Web :3000
```

3. **Ou séparément (deux terminaux)**

```
pnpm api:dev                          # terminal 1 — API :4000
pnpm --filter @cosmoz/web dev         # terminal 2 — Web :3000
```

4. **Ouvrir l'application**
   Web : **http://localhost:3000** · API : **http://localhost:4000** (préfixe `/api/v1`). Le premier `next dev` compile à la volée : laissez 10-30 s.

SECTION 8

Premier accès & création de compte

1. Ouvrez **http://localhost:3000** — écran d'authentification (groupe de routes `(auth)`).
2. Route **/register** : email + mot de passe fort. Hachage **Argon2id** (paramètres OWASP), anti-énumération d'emails, rate limiting multi-couches.
3. Email de vérification : en local sans SMTP, le mailer journalise le lien dans la console API — copiez-le dans le navigateur.
4. Connectez-vous, puis activez la **MFA (TOTP)** dans les réglages sécurité (secrets chiffrés AES-GCM, codes de récupération hachés à usage unique).
5. La base est vide : créez d'abord vos **comptes de trading** (mock provider activé en dev ; OANDA/Deriv en opt-in avec vos clés API), puis alimentez les autres modules.

SECTION 9

Vérifier que tout fonctionne

```
# API up
curl -s http://localhost:4000/api/v1/health | head -c 200
# Web up (proxy /api/v1 vers l'API)
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000
# Base up + batterie de tests
pnpm --filter @cosmoz/api db:status
pnpm test
```

Suite (référence 2026-10-19)	Résultat	Notes
Web (vitest)	**223/223**	~7 s
API (vitest, Postgres réel)	**1350/1351**	1 flake pré-existant : `oanda recovery sweep` (timeout 30 s, timing scheduler) — non bloquant
TypeScript	0 erreur	`tsc --noEmit` ×3 (web, api, shared)
Gardes	PASS	anti-mocks prod + anti-secrets trackés (1528 fichiers)
`pnpm audit --prod`	**0 vulnérabilité**	3 HIGH transitives épinglées via overrides

SECTION 10

Services optionnels

OCR — reconnaissance de captures (FastAPI, :3004)

```
cd mini-services/ocr-service
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --host 127.0.0.1 --port 3004
```

Puis dans `apps/api/.env` : `OCR_SERVICE_URL=http://127.0.0.1:3004` et `OCR_SERVICE_TOKEN=<randomBytes(32) hex>` (redémarrez l'API). Sans ce service, le module OCR dégrade proprement (`OCR_UNAVAILABLE`) — l'app reste fonctionnelle. Uploads validés par magic bytes.

Market data sidecar (Bun, :3003)

```
cd mini-services/market-service && bun run dev
# puis dans apps/api/.env : MARKET_SERVICE_URL=http://localhost:3003
```

SECTION 11

Build de production local

```
pnpm build   # turbo build : next build + nest build
```

- `NODE_ENV=production` **désactive** mock trading provider et mock OCR — une vraie `OCR_SERVICE_URL` devient requise.
- `CORS_ORIGINS` doit lister vos origines exactes (`*` rejeté).
- Erreurs 5xx masquées côté client, aucun stack trace ne fuit ; headers de sécurité posés par le web (XFO, nosniff, referrer, CSP frame-ancestors).
- `TRUST_PROXY=true` uniquement derrière un reverse proxy HTTPS de confiance.

SECTION 12

Structure du dépôt

```
oryon/
├── apps/
│   ├── web/                  # Frontend Next.js
│   │   ├── src/app/(auth)/   #   login, register, MFA
│   │   ├── src/app/(app)/    #   modules métier (accounts, analytics,
calendar…)
│   │   └── scripts/          #   garde anti-mocks prod
│   └── api/                  # Backend NestJS
│       ├── src/modules/      #   25+ modules métier
│       ├── src/infrastructure/ # health, env, sécurité
│       ├── prisma/           #   schema + 42 migrations (structure seule)
│       └── scripts/dev-db.mjs  # PostgreSQL embarquée
├── packages/shared/          # Contrats Zod, primitives monétaires, utils CSV
├── mini-services/            # ocr-service (FastAPI) + market-service (Bun)
├── docs/specs/master/        # Spécifications phases
├── docs/audits/              # 20+ rapports d'audit + CE GUIDE
├── docker-compose.yml        # db+api loopback-only (127.0.0.1)
├── pnpm-workspace.yaml       # overrides sécurité
└── turbo.json                # Orchestration dev/build/test
```

SECTION 13

Dépannage

Symptôme	Cause	Solution
L'API ne démarre pas (erreur env)	Variable obligatoire absente	Comparez avec `apps/api/.env.example` ; 4 clés + `DATABASE_URL` requises
`ECONNREFUSED 127.0.0.1:5433`	DB embarquée arrêtée	`pnpm api:db:up` puis relancez l'API

`P1003` / migrations manquantes	Schéma non appliqué	`prisma migrate deploy`
Le web affiche des données fictives	Mode MSW actif	`NEXT_PUBLIC_API_MOCKED=false` dans `.env.local`, redémarrez
`404 /api/v1/...` depuis le web	API arrêtée / mauvais proxy	Vérifiez API :4000 et `API_PROXY_TARGET`
Erreur CORS	Origine non allowlistée	Ajoutez l'origine exacte à `CORS_ORIGINS`
Port 3000/4000 occupé	Autre service local	Libérez le port ou changez la config
`pnpm: command not found`	pnpm absent	`corepack enable && corepack prepare pnpm@12.4.2 --activate`
1 échec test `oanda recovery sweep`	Flake de timing connu	Relancez — documenté non bloquant
Reset complet de la base	Repartir de zéro	`db:down` → suppr. `.localpg` → `db:up` → `migrate deploy`

SECTION 14

Sécurité & bonnes pratiques

1. **Secrets** : régénérez les 4 clés si vous exposez l'app au-delà de localhost. Ils ne quittent jamais votre machine.
2. **Fichiers .env** : exclus du dépôt et surveillés par la garde CI — ne forcez jamais `git add -f apps/api/.env`.
3. **Mots de passe** : Argon2id côté serveur ; n'abaissez jamais les paramètres.
4. **MFA** : activation recommandée dès le premier login.
5. **Providers réels** (OANDA/Deriv) : opt-in explicite, clés chiffrées AES-256-GCM au repos, jamais retournées par l'API.
6. **Webhooks entrants** : HMAC + fenêtre anti-replay intégrés.
7. **Mises à jour** : `pnpm audit --prod` doit rester à 0 ; respectez les `overrides` de `pnpm-workspace.yaml`.
8. **Backups** : `pg_dump` + copie de `apps/api/.env` hors machine — c'est votre seul état irremplaçable une fois vos données saisies.

Prêt à trader
avec vos données.

- ✓ pnpm 12.4.2 installé — `corepack prepare pnpm@12.4.2 --activate`
- ✓ `pnpm install --frozen-lockfile` à la racine
- ✓ `apps/api/.env` : 4 clés CSPRNG + `DATABASE_URL`
- ✓ `apps/web/.env.local` : `NEXT_PUBLIC_API_MOCKED=false`
- ✓ `pnpm api:db:up` puis `prisma migrate deploy` — base vide
- ✓ `pnpm dev` → http://localhost:3000
- ✓ Premier compte via /register, puis MFA recommandée